Infrastructure Automation using Terraform, Ansible & Jenkins CI/CD on AWS

Project Overview

This project demonstrates how to automate AWS infrastructure creation and server configuration using **Terraform**, **Ansible**, and **Jenkins**.

Terraform is used to create AWS resources such as a **VPC, Subnet, Internet Gateway, Route Table, Security Group, and EC2 instance**. After the infrastructure is created, **Ansible** automatically connects to the EC2 instance and installs and configures the **Nginx web server**.

The entire process is automated using a **Jenkins CI/CD pipeline**. Whenever the pipeline runs, Jenkins pulls the latest code from GitHub, executes Terraform commands to provision the infrastructure, and then runs the Ansible playbook to configure the server without manual intervention.

This project demonstrates the **Infrastructure as Code (IaC)** approach, making deployments faster, consistent, and easier to manage.

Definition

This project is an **Infrastructure as Code (IaC)** and **Configuration Management** project that uses **Terraform**, **Ansible**, and **Jenkins** to automatically create AWS infrastructure and configure a web server.

Goal of the Project

The main goal of this project is to:

- Automate AWS infrastructure creation
- Configure EC2 automatically using Ansible
- Install and configure Nginx
- Build a CI/CD pipeline using Jenkins
- Reduce manual work and deployment time

Scope

This project focuses on automating infrastructure provisioning and server configuration on AWS using DevOps tools. It covers the complete workflow from infrastructure creation to application deployment through a Jenkins CI/CD pipeline.

Project Scope

- Provision AWS infrastructure using Terraform.
- Create VPC, Subnet, Internet Gateway, Route Table, Security Group, and EC2 instance.
- Configure the EC2 instance automatically using Ansible.
- Install and configure the Nginx web server.
- Automate the deployment process using Jenkins.
- Manage project source code with Git and GitHub.
- Demonstrate Infrastructure as Code (IaC) and configuration management.

AWS Services Used

1. Amazon EC2
Purpose: Used to create a virtual server for hosting the web application.
2. Amazon VPC
Purpose: Creates a private virtual network for AWS resources.
3. Subnet
Purpose: Provides a network for the EC2 instance inside the VPC.
4. Internet Gateway
Purpose: Allows internet access to the VPC.
5. Route Table
Purpose: Routes network traffic between the subnet and the Internet Gateway.
6. Security Group
Purpose: Controls inbound and outbound traffic for the EC2 instance.

Tools Used

1. Terraform
Purpose: Automates the creation of AWS infrastructure.
2. Ansible
Purpose: Installs and configures Nginx on the EC2 instance automatically.
3. Jenkins
Purpose: Automates the complete CI/CD pipeline.
4. Git
Purpose: Tracks and manages source code changes.
5. GitHub
Purpose: Stores the project code and Jenkins pipeline files.
6. Nginx
Purpose: Works as the web server to host the website.
7. Linux (Amazon Linux 2023)
Purpose: Operating system used for the Jenkins server and the target EC2 instance.

Architecture

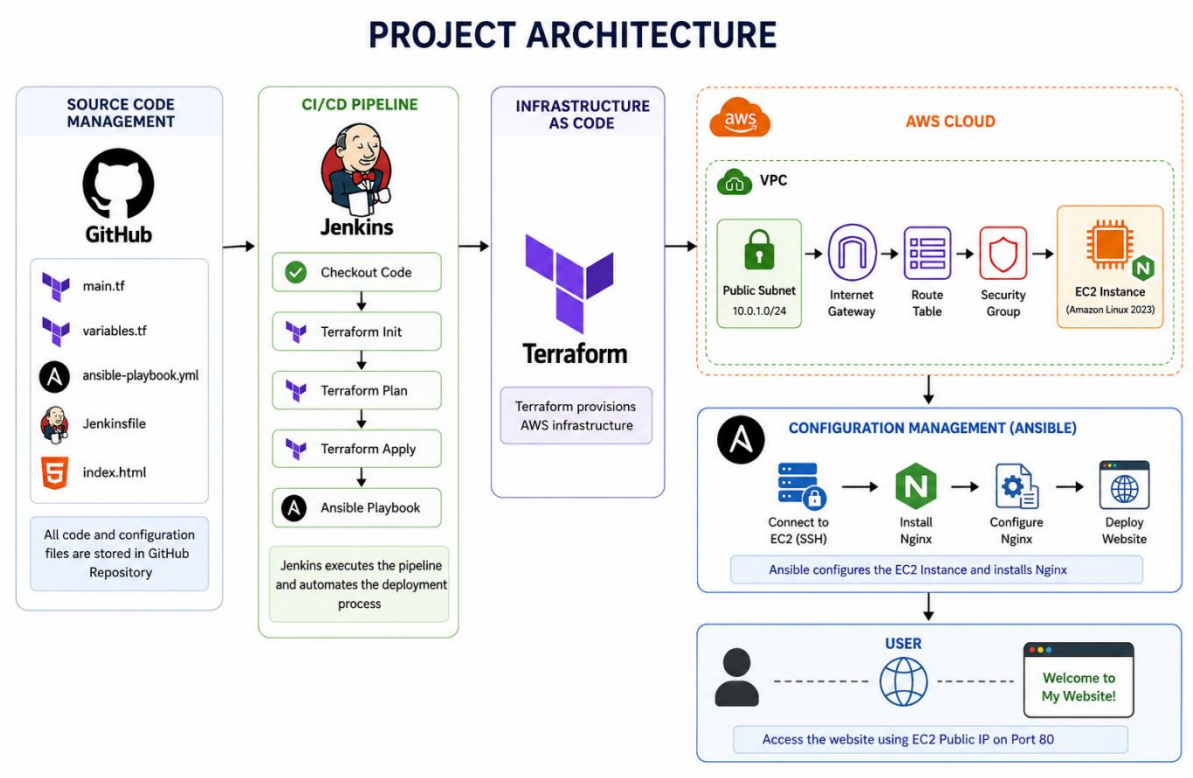
This project follows an automated Infrastructure as Code (IaC) workflow using **GitHub**, **Jenkins**, **Terraform**, **AWS**, and **Ansible**.

The project starts with the source code stored in GitHub. Jenkins executes the CI/CD pipeline by checking out the latest code from the repository. Terraform then provisions the required AWS infrastructure, including the VPC, Subnet, Internet Gateway, Route Table, Security Group, and EC2 instance. Once the infrastructure is created, Ansible connects to the EC2 instance through SSH and installs and

configures the Nginx web server. After the deployment is completed, the website becomes accessible through the EC2 instance's public IP address.

Workflow:

GitHub Repository
↓
Jenkins CI/CD Pipeline
↓
Terraform (Infrastructure Provisioning)
↓
AWS Infrastructure (VPC, Subnet, Internet Gateway, Route Table, Security Group, EC2)
↓
Ansible (Server Configuration)
↓
Nginx Installation & Website Deployment
↓
Website Accessible via EC2 Public IP



Project Folder Structure

terraform-ansible-project/
|

```
├── Jenkinsfile
├── README.md
│
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── terraform.tfvars
│   └── provider.tf
│
├── ansible/
│   ├── inventory
│   ├── playbook.yml
│   └── files/
│       └── index.html
│
└── website/
    └── index.html
```

Step-by-Step Implementation

Step 1 – Launch Jenkins Server (EC2)

Created an Amazon EC2 instance to set up the DevOps environment.

Configuration

Setting	Value
AMI	Amazon Linux 2023
Instance Type	t3.small
Region	ap-south-1
Storage	20 GB
Key Pair	aws-key

Purpose

- Host Jenkins
- Install Terraform and Ansible
- Execute the CI/CD pipeline

Step 2 – Install Required Tools

Installed all required tools on the Jenkins server.

Installed Tools

- Git
- Java 21
- Jenkins
- Terraform
- Ansible

- AWS CLI

```
[root@ip-172-31-7-18 terraform-ansible-project]# # Check Git version
git --version

# Check Java version
java -version

# Check Jenkins version
jenkins --version
# If this doesn't work:
rpm -qi jenkins

# Check Terraform version
terraform -version

# Check Ansible version
ansible --version
git version 2.50.1
```

Purpose

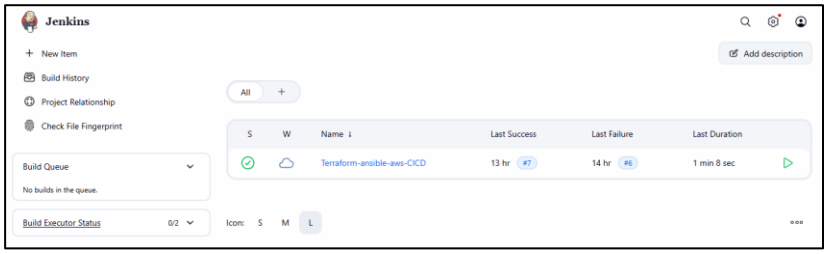
These tools are required to automate infrastructure provisioning, configuration management, and CI/CD.

Step 3 – Configure Jenkins

Configured Jenkins after installation.

Tasks Performed

- Started Jenkins service
- Enabled Jenkins to start automatically
- Accessed Jenkins using port **8080**
- Completed the initial setup
- Installed the recommended plugins
- Created the administrator account



Purpose

Prepare Jenkins to automate the deployment pipeline.

Step 4 – Create Project Directory

Created the project directory.

mkdir terraform-ansible-project

Created the project structure.

```
[root@ip-172-31-7-18 terraform-ansible-project]# cd ..
[root@ip-172-31-7-18 ec2-user]# tree
.
└── terraform-ansible-project
    ├── Jenkinsfile
    ├── README.md
    ├── ansible
    │   ├── ansible.cfg
    │   ├── files
    │   │   └── index.html
    │   ├── inventory
    │   └── playbook.yml
    └── terraform
        ├── main.tf
        ├── outputs.tf
        ├── terraform.tfstate
        ├── terraform.tfstate.backup
        ├── terraform.tfvars
        └── variables.tf

4 directories, 12 files
[root@ip-172-31-7-18 ec2-user]#
```

Purpose

Organize Terraform, Ansible, and Jenkins files in a single project.

Step 5 – Create Terraform Variables

Created the following files:

- variables.tf

```
variable "region" {
  description = "AWS Region"
  type        = string
}

variable "vpc_cidr" {
  description = "VPC CIDR Block"
  type        = string
}

variable "public_subnet_cidr" {
  description = "Public Subnet CIDR Block"
  type        = string
}

variable "availability_zone" {
  description = "Availability Zone"
  type        = string
}

variable "instance_type" {
  description = "EC2 Instance Type"
  type        = string
}
```

```
variable "instance_type" {
  description = "EC2 Instance Type"
  type        = string
}

variable "key_name" {
  description = "AWS Key Pair Name"
  type        = string
}

variable "ami_id" {
  description = "Amazon Linux AMI ID"
  type        = string
}
```

- terraform.tfvars

```
region             = "ap-south-1"
vpc_cidr           = "10.0.0.0/16"
public_subnet_cidr = "10.0.1.0/24"
availability_zone  = "ap-south-1a"
instance_type      = "t3.small"
key_name           = "aws-key"
ami_id             = "ami-01a18c38ece67e620"
```

Purpose

Store infrastructure variables separately from the Terraform code, making the project reusable and easier to manage.

Step 6 – Create Terraform Configuration

Created the **main.tf** file to define the AWS infrastructure.

Resources Created

- Provider

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.region
}
```

- VPC
- Public Subnet
- Internet Gateway

```
# VPC

resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

  tags = {
    Name = "Project-VPC"
  }
}

# ------------------------
# Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Project-IGW"
  }
}
```

```
# --------------------------
# Public Subnet

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidr
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "Public-Subnet"
  }
}
```

-
- Route Table
- Route Table Association

```
# Route Table
# --------------------------
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Public-RouteTable"
  }
}


# Route Table Association
# -------------------------
resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public_rt.id
}
```

- Security Group

```
# Security Group
# --------------------------
resource "aws_security_group" "web_sg" {
  name        = "web-security-group"
  description = "Allow SSH and HTTP"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
```

- EC2 Instance

```
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Web-SG"
  }
}

# EC2 Instance
# --------------------------
resource "aws_instance" "web" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  key_name               = var.key_name

  tags = {
    Name = "Terraform-Ansible-Server"
  }
```

- Outputs (Instance ID, Public IP, VPC ID)

```
output "instance_id" {
  description = "EC2 Instance ID"
  value       = aws_instance.web.id
}

output "instance_public_ip" {
  description = "EC2 Public IP"
  value       = aws_instance.web.public_ip
}

output "vpc_id" {
  description = "VPC ID"
  value       = aws_vpc.main.id
}
```

Purpose: Define the complete AWS infrastructure using Infrastructure as Code (IaC) so it can be created automatically with Terraform.

tep 7 – Initialize Terraform

Ran the following command: terraform init

Purpose: Initialize the Terraform working directory and download the required AWS provider plugins.

Step 8 – Validate the Configuration

Executed: terraform validate

Purpose: Verify that the Terraform configuration is correct before deployment.

Step 9 – Generate the Execution Plan

Executed: terraform plan

Purpose: Preview the AWS resources that Terraform will create without making any changes.

Step 10 – Create the Ansible Configuration

Inside the **ansible** folder, created:

- inventory

```
[root@ip-172-31-7-18 ansible]# cat inventory
[web]
43.205.114.205

[web:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=/var/lib/jenkins/.ssh/aws-key.pem
[root@ip-172-31-7-18 ansible]#
```

- playbook.yml

```
[root@ip-172-31-7-18 ansible]# cat playbook.yml
---
- name: Install and Configure Nginx
  hosts: web
  become: yes

  tasks:

    - name: Install Nginx
      dnf:
        name: nginx
        state: present

    - name: Start Nginx Service
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Copy Website
      copy:
        src: files/index.html
        dest: /usr/share/nginx/html/index.html
```

- index.html

```
[root@ip-172-31-7-18 files]# cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>Terraform + Ansible Project</title>
</head>
<body>
    <h1>Terraform + Ansible Automation</h1>
    <h2>Created by Mudasir Yousuf</h2>
    <p>This web server was configured automatically using Ansible.</p>
</body>
</html>
```

Purpose

- Inventory stores the target EC2 server details.
- Playbook automates Nginx installation and website deployment.
- index.html is the sample web page deployed to the server.

Step 11 – Test Ansible Connection

Verified the connection between the Jenkins server and the target EC2 instance.

Command

ansible all -m ping

```
43.205.114.205 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
```

Purpose: Ensure that Ansible can successfully connect to the EC2 instance before running the playbook.

Step 12 – Create the Jenkins Pipeline

Created a **Jenkins Pipeline** project and connected it to the GitHub repository.

Pipeline Stages

- Checkout
- Terraform Init
- Terraform Validate
- Terraform Plan
- Ansible Ping
- Ansible Deploy
- Terraform Apply

```
[root@ip-172-31-7-18 terraform-ansible-project]# cat Jenkinsfile
pipeline {
    agent any

    environment {
        AWS_DEFAULT_REGION = 'ap-south-1'
    }

    stages {

        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                dir('terraform') {
                    withCredentials([[
                        $class: 'AmazonWebServicesCredentialsBinding',
                        credentialsId: 'aws'
                    ]]) {
                        sh 'terraform init'
                    }
                }
            }
        }

        stage('Terraform Validate') {
```

```
stage('Terraform Validate') {
    steps {
        dir('terraform') {
            sh 'terraform validate'
        }
    }
}

stage('Terraform Plan') {
    steps {
        dir('terraform') {
            withCredentials([[
                $class: 'AmazonWebServicesCredentialsBinding',
                credentialsId: 'aws'
            ]]) {
                sh 'terraform plan'
            }
        }
    }
}

stage('Ansible Ping') {
    steps {
        dir('ansible') {
            sh 'ansible all -m ping'
        }
    }
}

stage('Ansible Deploy') {
```

```
        stage('Ansible Deploy') {
            steps {
                dir('ansible') {
                    sh 'ansible-playbook playbook.yml'
                }
            }
        }

        stage('Terraform Apply') {
            steps {
                dir('terraform') {
                    withCredentials([[
                        $class: 'AmazonWebServicesCredentialsBinding',
                        credentialsId: 'aws'
                    ]]) {
                        sh 'terraform apply -auto-approve'
                    }
                }
            }
        }
    }
}
```

PurposeAutomate the complete infrastructure provisioning and server configuration process.

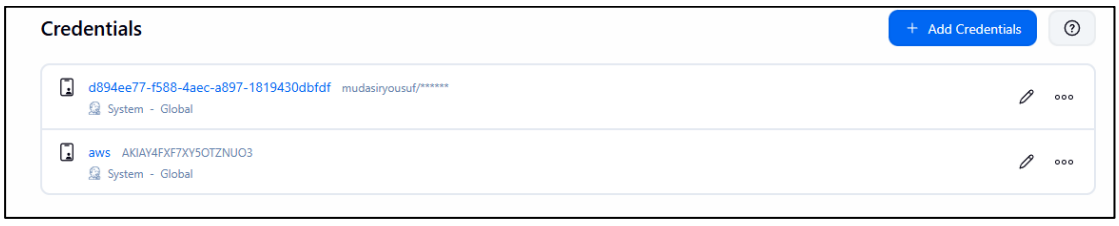
Step 13 – Configure Jenkins Credentials

Added the required credentials in Jenkins.

Credentials Added

- AWS Access Key
- AWS Secret Access Key
- GitHub Credentials

Purpose: Allow Jenkins to securely access AWS resources and the GitHub repository.



Step 14 – Create the Jenkinsfile

Created the Jenkinsfile in the project repository.

Purpose: Define all CI/CD pipeline stages as code so Jenkins can automatically execute the workflow.

Step 15 – Push Project to GitHub

Uploaded the complete project to GitHub.

Commands Used

git initgit add .git commit -m "Initial Commit"git branch -M maingit remote add origin <repository-url>git push -u origin main

Purpose: Store the project in a GitHub repository and enable Jenkins to fetch the latest code automatically.

Step 16 – Run the Jenkins Pipeline

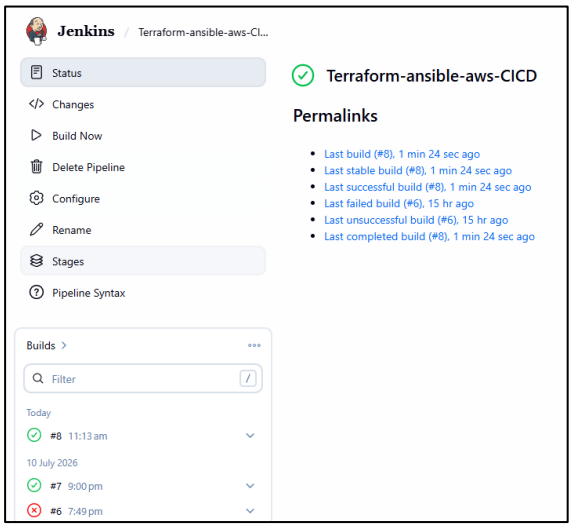
Triggered the Jenkins pipeline using **Build Now**.

Pipeline Workflow

Checkout
↓
Terraform Init
↓
Terraform Validate
↓
Terraform Plan
↓
Ansible Ping
↓
Ansible Deploy
↓
Terraform Apply

Purpose: Execute the complete automation process from infrastructure provisioning to server configuration.

```
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



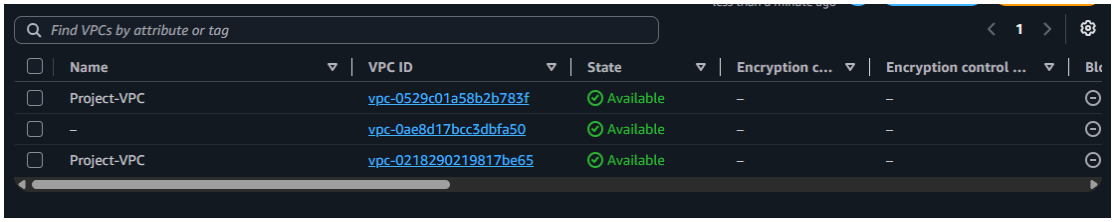
Step 17 – Verify Infrastructure

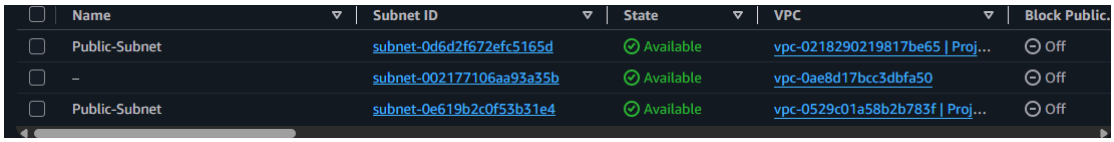
After the pipeline completed successfully, verified the resources created in AWS.

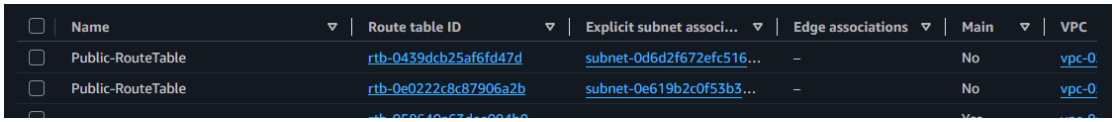
Resources Verified

- VPC
- Public Subnet
- Internet Gateway
- Route Table
- Security Group
- EC2 Instance

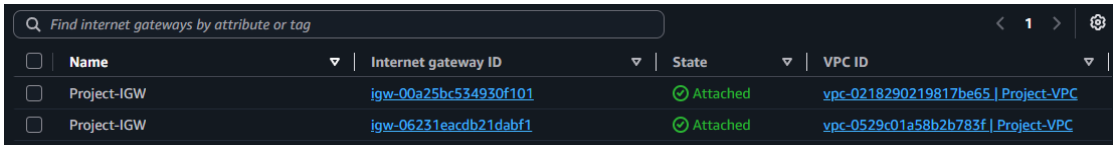
PurposeConfirm that Terraform successfully provisioned the required AWS infrastructure.





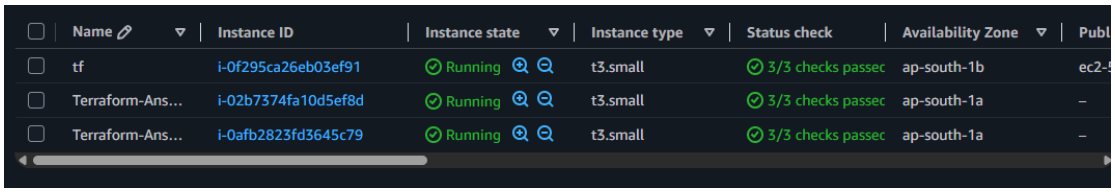


Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
Public-RouteTable	rtb-0439dcb25af6fd47d	subnet-0d6d2f672efc516...	–	No	vpc-0
Public-RouteTable	rtb-0e0222c8c87906a2b	subnet-0e619b2c0f53b3...	–	No	vpc-0



Find internet gateways by attribute or tag

Name	Internet gateway ID	State	VPC ID
Project-IGW	igw-00a25bc534930f101	Attached	vpc-0218290219817be65 | Project-VPC
Project-IGW	igw-06231eacdb21dabf1	Attached	vpc-0529c01a58b2b783f | Project-VPC



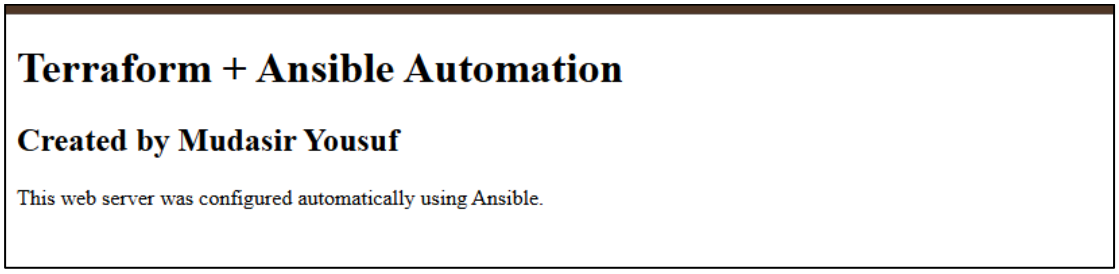
Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Publ
tf	i-0f295ca26eb03ef91	Running	t3.small	3/3 checks passec	ap-south-1b	ec2-5
Terraform-Ans...	i-02b7374fa10d5ef8d	Running	t3.small	3/3 checks passec	ap-south-1a	–
Terraform-Ans...	i-0afb2823fd3645c79	Running	t3.small	3/3 checks passec	ap-south-1a	–

Step 18 – Verify Nginx Deployment

Accessed the EC2 instance's **Public IP** in a web browser.

Result: The default web page deployed by Ansible was displayed successfully.

Purpose: Verify that Ansible successfully installed and configured the Nginx web server.



Terraform + Ansible Automation

Created by Mudasir Yousuf

This web server was configured automatically using Ansible.

Project Workflow

Developer

Push Code to GitHub

Jenkins Pulls Latest Code

Terraform Init

Terraform Validate

Terraform Plan

Skills Learned

- Infrastructure as Code (IaC)
- Terraform basics and AWS resource provisioning
- Ansible configuration management
- Jenkins CI/CD pipeline creation
- AWS networking (VPC, Subnet, Route Table, Internet Gateway)
- EC2 instance provisioning
- Security Group configuration
- Git and GitHub integration
- Linux server administration
- Nginx installation and configuration
- Automation of infrastructure deployment

Troubleshooting

Issue 1 – Terraform Authentication Failed

Cause: AWS credentials were not configured correctly in Jenkins.

Solution: Configured AWS credentials in Jenkins using the Credentials Manager and used them in the Jenkins pipeline.

Issue 2 – Ansible Could Not Connect to EC2

Cause: SSH private key path and permissions were incorrect.

Solution: Copied the private key to the Jenkins user, updated the inventory file, and set the correct file permissions.

Issue 3 – Jenkins Could Not Read Inventory File

Cause: Jenkins did not have permission to access the inventory file.

Solution: Used the inventory file inside the Jenkins workspace and updated the required file permissions

Conclusion

This project successfully demonstrates Infrastructure as Code (IaC) by using Terraform, Ansible, and Jenkins on AWS. Terraform automatically creates the AWS infrastructure, Ansible configures the EC2 instance by installing Nginx, and Jenkins automates the complete deployment process through a CI/CD pipeline. This project reduces manual work, improves consistency, and provides a simple and efficient way to provision and configure cloud infrastructure.